

GreenPulse

Public Park Ecosystem Health Monitoring System & Urban Biodiversity Mapping Portal

Every section, every tab, every feature - and the technology behind each.

This guide explains what the project does and how it is built, section by section. It exists so that you can explore the system confidently and answer questions about it without having written every line yourself.

Each module chapter follows the same shape: what the section is for, what each tab does, exactly which technology and which file produces it, and the questions an examiner is most likely to ask about it.

Read this first if you have five minutes

Chapter 2 (Tech Stack) and Chapter 4 (Dashboard) together cover most of what a viva will ask. Chapter 16 is a one-page cheat sheet. Chapter 15 lists what the project deliberately does NOT do - knowing your own limitations is the single strongest thing you can demonstrate in a review.

At a glance	
Modules	12, plus authentication and external integrations
Frontend	Next.js 13.5 (App Router), TypeScript, Tailwind, shadcn/ui - 16 routes
Backend	Node.js + Express 4, Mongoose 8, JWT - 134 route handlers
Database	MongoDB, 16 collections, GeoJSON with 2dsphere indexes
Public APIs	Open-Meteo (weather + air quality), GBIF, OpenStreetMap - no API key needed
Code size	approx. 29,000 lines across 179 source files

Contents

1. How to Run It	2
2. The Technology Stack	4
3. How the Pieces Fit Together	6
4. Ecosystem Monitoring Dashboard	8
5. GIS & Urban Biodiversity Mapping	9
6. Park Asset Management	10
7. Biodiversity Management	11
8. AI Ecosystem Monitoring	13
9. Environmental Sensor Monitoring	14
10. Citizen Engagement Portal	16
11. Incident & Alert Management	17
12. Maintenance Management	18
13. Analytics & Reports	19
14. AI Environmental Assistant	20
15. Administration	21
16. The Mathematics, in Plain English	22
17. The Database	25
18. What This Project Does Not Do	26
19. One-Page Cheat Sheet	27

1. How to Run It

The project is two separate applications that talk over HTTP. Start the backend first, because the frontend calls it on load. You need two terminal windows.

Terminal 1 - backend (the API)

```
cd GreenPulse---Escosystem\backend
npm install          # first time only
npm run dev

# wait for: GreenPulse API listening on http://localhost:5000
```

Terminal 2 - frontend (the website)

```
cd GreenPulse---Escosystem\frontend
npm install          # first time only
npm run build        # do this before a presentation
npm start

# then open http://localhost:3000
```

Use "npm run build && npm start" for the review, not "npm run dev"

In dev mode Next.js compiles each page the first time you visit it, which takes 5-18 seconds and looks exactly like a hang. The production build compiles everything up front, and pages then load in under 50 milliseconds. Use dev mode only while editing code.

Signing in

The login page has one-click buttons for all four roles, so you do not need to type anything. Every account uses the password greenpulse123.

Account	Can do
Officer officer@greenpulse.gov	Incidents, work orders, assets, sensors, reviewing citizen reports. Best account for a demo.
Admin admin@greenpulse.gov	Everything, plus user management, system settings and the audit log.
Ecologist ecologist@greenpulse.gov	Species catalogue and verifying citizen wildlife sightings.
Citizen citizen@greenpulse.gov	Submitting reports and sightings, viewing own contributions.

The dashboard, map and biodiversity pages are public and work without signing in at all. That is deliberate: transparency to the public is one of the stated benefits of the project.

If something goes wrong

- **"Cannot reach the API" on every page** - the backend is not running. Start Terminal 1. The error message on screen tells you the exact command.
- **"Live data unavailable" in the Conditions panel** - you are offline. Everything else still works; the sensors fall back to their own model. Say so if asked, it is designed behaviour.
- **Data looks wrong mid-demo** - sign in as admin, go to Administration, System tab, and press Reseed. Takes about ten seconds and restores a known-good dataset.

- **Port already in use** - another copy is already running. Either use it, or close the other terminal.

2. The Technology Stack

This chapter is the one to read before a viva. For each library it says what the thing is, what it does in this project specifically, and why it was chosen over the obvious alternative. Examiners rarely ask "what is React"; they ask "why did you use this".

Frontend

Technology	What it is, and what it does here
Next.js 13.5 (App Router)	A React framework. It provides file-based routing (a folder under app/ becomes a URL), server-side rendering for fast first paint, and a production build step. Chosen over plain React because routing, bundling and SSR come built in rather than being assembled by hand.
React 18	The UI library. Everything on screen is a React component that re-renders when its data changes.
TypeScript 5.6	JavaScript with static types. Every API response has a declared shape in lib/types.ts, so if the backend changes a field name the frontend fails to compile instead of silently showing "undefined" to a user.
Tailwind CSS	Utility-first styling. Classes like "flex gap-2 rounded-lg" are composed directly in the markup. Chosen because it keeps styles next to the element they affect, so nothing breaks elsewhere when you change one.
shadcn/ui (+ Radix UI)	Accessible component primitives - dialogs, dropdowns, tabs, sliders. Not an npm dependency: the source is copied into components/ui/ so it can be edited. Radix supplies the behaviour (keyboard navigation, focus trapping, ARIA); Tailwind supplies the appearance.
TanStack Query v5	Server-state management. Handles caching, background refetching, loading and error states, and cache invalidation after a write. Without it every page would need its own useState/useEffect fetch logic with the same bugs repeated.
React Hook Form + Zod	Forms and validation. Zod describes the rules once; the same schema shape is used on the server, so the browser and the API agree on what valid input is.
Leaflet + React-Leaflet	The interactive map. Open-source, no API key and no usage limits - the reason it was chosen over Google Maps or Mapbox.
Recharts	All charts. Built on SVG and composable as React components, so a chart is written the same way as any other part of the UI.
jsPDF + autoTable	Client-side PDF export in the Analytics module. Generating PDFs in the browser avoids shipping a headless browser into the server deployment.
Framer Motion	Animation - the sidebar drawer, the health gauge sweep, card entrances.
next-themes	Dark and light mode, persisted per browser.
Sonner	Toast notifications for the result of every write action.

Backend

Technology	What it is, and what it does here
Node.js 22	The JavaScript runtime the server runs on. Using JavaScript on both sides means one language for the whole project.
Express 4	The web framework. Defines the routes, applies middleware in order (security, CORS, rate limiting, authentication, validation), and hands off to controllers.

Technology	What it is, and what it does here
MongoDB	The database. A document store: a park is one document containing its geometry, scores and facilities, rather than being split across five tables. Chosen over SQL because the data is naturally nested and because GeoJSON is a first-class citizen.
Mongoose 8	The ODM (object-document mapper) over MongoDB. Provides schemas, validation, indexes, population of references, and pre-save hooks - for example, an asset's text status is derived from its numeric condition automatically so the two can never disagree.
mongodb-memory-server	Runs a real MongoDB in memory when no connection string is configured. This is why the project runs on any machine with no database installed - a deliberate choice so a marker never has to install anything.
JWT (jsonwebtoken)	Authentication. On login the server returns a signed token; the browser sends it back on every request. Stateless, so the server keeps no session store.
bcryptjs	Password hashing. Passwords are never stored or logged in plain text.
Zod	Request validation on every write route. Rejects bad input with per-field messages before it reaches the database.
Helmet, CORS, express-rate-limit	Security middleware: sets protective HTTP headers, restricts which origins may call the API, and throttles requests. The login route has a stricter limit than everything else because it is the only endpoint worth brute-forcing.

Public APIs used (none require an API key)

Service	What it provides
Open-Meteo Forecast	Live weather and a 7-day forecast for each park's real coordinates. Feeds the Conditions panel and anchors the sensor simulator.
Open-Meteo Air Quality (CAMS)	Raw pollutant concentrations - PM2.5, PM10, NO2, SO2, CO, O3. Important: it returns concentrations, not a finished index, so this project computes the AQI itself using CPCB breakpoints. The formula is genuinely being applied to real measurements.
GBIF	The Global Biodiversity Information Facility. Used to cross-check a catalogued species against the world occurrence record: if GBIF has no record of that species within 50 km, the local identification is flagged for review.
OpenStreetMap Nominatim	Reverse geocoding - converts the coordinates of a citizen report into a readable street address.
OpenWeatherMap, eBird	Optional alternatives. They activate automatically if an API key is supplied, and are simply skipped otherwise.

Why keyless services were chosen

A project that only works once the marker has registered for four accounts is not a working project. Every call also has an 8-second timeout and a local fallback, so the entire system still functions with no internet connection at all - the data is simply generated rather than anchored to reality.

3. How the Pieces Fit Together

The request path

Every single action in the application follows this path. If you can recite it, you can answer most architecture questions.

```
Browser (React component)
  | calls a hook, e.g. useDashboard()
lib/hooks/use-api.ts      - TanStack Query: cache, loading, errors
  |
lib/api/endpoints.ts      - one function per API route
  |
lib/api/client.ts         - adds the JWT, unwraps the response envelope
  | HTTP
backend: src/app.js       - helmet, CORS, rate limit, JSON parsing
  |
src/routes/*.routes.js    - URL matching
  |
middleware/auth.js        - is the token valid? is the role high enough?
middleware/validate.js    - does the body match the Zod schema?
  |
src/controllers/*.js      - orchestration
  |
src/services/*.js         - the mathematics and business rules
  |
src/models/*.js           - Mongoose schemas
  |
MongoDB
```

Why the code is split this way

- **Routes only match URLs.** They contain no logic, so the full API surface can be read in a few minutes.
- **Controllers orchestrate.** They fetch, call services, and shape the response. They contain no formulas.
- **Services hold the thinking.** Every formula lives in a service, which is why the mathematics can be tested and explained on its own - and why it is not buried inside a route handler.
- **Models own the data rules.** Validation, indexes and derived fields live with the schema, so they apply no matter which code path writes.
- **The frontend never builds a URL.** It calls a typed function. Changing a route is a one-line edit rather than a search across a dozen files.

One response shape everywhere

Every endpoint answers in the same envelope, so the client unwraps it once rather than per endpoint.

```
success: { "success": true, "data": ..., "meta": { pagination } }
failure: { "success": false, "error": { "message": ..., "details": ... } }
```

The role hierarchy

Four roles, each including everything below it. A route marked "officer" therefore also admits an admin, which avoids listing every superior role on every route.

```
citizen (1) -> ecologist (2) -> officer (3) -> admin (4)
```

The interface hides what your role cannot use, but that is a courtesy, not the control. The API enforces the same rule

independently in `middleware/auth.js` - hiding a button does not stop anybody calling the endpoint directly, so the server has to check as well.

4. Ecosystem Monitoring Dashboard

/dashboard - Module 1 - public, no sign-in needed

What it is for

The landing page and the command centre. It answers "how are the parks doing right now" in one screen: a composite health score, the five sub-indices behind it, live weather and air quality, active alerts, the highest-priority incidents, a cross-module activity feed, and a ranking of parks worst-to-best.

Tabs

Tab	What it shows and why it matters
KPI row	Eight tiles - ecosystem health, biodiversity, air, water, soil, tree health, species count, active alerts. Each is coloured by its normalised score, so an AQI tile can read "84 AQI" while still being coloured by how good 84 actually is.
Health gauge	The composite index, plus a bar per sub-index showing its weighted contribution. This is what makes the score auditable: you can see which indicator is dragging it down and by how much.
Live conditions	Real weather and real air quality from public APIs, with the per-pollutant CPCB sub-indices shown so the calculation is visible rather than merely claimed.
Biodiversity card	Species composition by class, plus the actual index values (Shannon, evenness, Gini-Simpson) rather than just a single score.
Trends	Environmental indicators over 7, 30 or 90 days, all normalised to 0-100 so they share one axis.
Queues + alerts	Open incidents, reports awaiting review, work orders due. Each links to its module.
Park ranking	Every park scored side by side. The lowest bar is where budget should go first.

What powers it

Layer	Detail
Page	frontend/app/(app)/dashboard/page.tsx
API	GET /api/dashboard/overview, /trend, /activity + /integrations/park-conditions
Backend	controllers/dashboard.controller.js - one parallel fan-out so the whole page needs a single round trip
Services	ecosystem-score.service.js (the composite), biodiversity.service.js, aqi.service.js, external.service.js
Data	Reads across Park, Sensor, Observation, Incident, Alert, Asset, EcoReport

Likely viva questions

- **How is the ecosystem health score calculated?** A weighted mean of five sub-indices: $EHI = \frac{\sum(w_k * S_k)}{\sum(w_k)}$, with air 0.25, water 0.20, soil 0.15, tree 0.20, biodiversity 0.20. Dividing by the sum of weights means a park missing a sensor is scored on the indicators it does have, rather than being punished with a zero.
- **Why those weights?** Air is highest because it moves fastest and affects visitors most directly. Soil is lowest because it changes slowly and is least directly experienced. They are configurable in Administration, not hard-coded.
- **Why one endpoint instead of several?** The page needs eleven different figures. Eleven requests would mean eleven round trips and a page that assembles itself in stages; one parallel fan-out on the server returns it all at once.

5. GIS & Urban Biodiversity Mapping

/map - Module 2 - public

What it is for

An interactive map of the city with eight switchable layers. The important design point: this module has no database collection of its own. Every layer is a live projection of data another module owns - a tree pin IS the asset register's record of that tree - which is why the map can never go stale relative to the rest of the system.

Tabs

Tab	What it shows and why it matters
Layers	Parks (with boundaries), trees and plants, water bodies, wildlife sightings, pollution hotspots, trails, sensors, citizen reports. Each chip shows a live feature count, so "layer is off" is distinguishable from "layer is on but empty".
Search	Matches across every visible layer. Non-matching features are dimmed rather than hidden, because removing them would destroy the spatial context that makes a match meaningful.
Pollution circles	Sized by the incident's computed priority score - the biggest circle is literally the one to deal with first.
Selection panel	Full record for whatever you click, with condition and health shown as bars rather than bare numbers.

What powers it

Layer	Detail
Page	frontend/app/(app)/map/page.tsx + components/map/biodiversity-map.tsx
API	GET /api/gis/layers, /gis/heatmap, /gis/within
Backend	controllers/gis.controller.js - composes GeoJSON FeatureCollections from other collections
Library	Leaflet 1.9 + React-Leaflet 4; OpenStreetMap, Esri satellite and OpenTopoMap tiles
Data	GeoJSON with 2dsphere indexes on Park, Asset, Observation, Incident, Sensor, CitizenReport

Likely viva questions

- **Why GeoJSON and not two number columns?** GeoJSON is the standard format Leaflet consumes directly, and MongoDB's 2dsphere index understands it - which is what makes "everything within 500 metres of this point" a single indexed query instead of fetching every row and filtering in code.
- **What is the coordinate trap here?** GeoJSON orders coordinates [longitude, latitude]; Leaflet expects [latitude, longitude]. Getting it backwards puts a park in the Indian Ocean and the bug is invisible until the map renders. The conversion is isolated in lib/api/geo.ts and never done inline.
- **Why does the map have no collection of its own?** Because duplicating tree positions into a "map features" table would mean two sources of truth that drift apart. Deriving the layers guarantees the map agrees with the asset register.

6. Park Asset Management

/assets - Module 3 - officer role to edit

What it is for

The digital inventory of every physical thing in a park: trees, plants, benches, lakes, paths, lights and structures. Each asset carries a 0-100 condition, a maintenance history, and type-specific attributes. Asset condition feeds the tree-health sub-index, so this module directly affects the headline ecosystem score.

Tabs

Tab	What it shows and why it matters
Summary tiles	Total assets, mean condition, how many need attention (condition below 50), and total maintenance spend.
Condition by type	Count and mean condition per asset type, coloured by condition. The weakest class is where a scheduled maintenance cycle pays off most.
Inventory table	Filter by type, status and park; search by name, code or notes. Condition is a bar, not a number, so a bad asset is visible at a glance.
Detail drawer	Attributes, maintenance history with costs, and any related work orders. Warns when condition is below 50 and explains the knock-on effect on the park score.
Log maintenance	Records work against an asset and raises its condition - because logging a repair without reflecting its effect would leave the register permanently pessimistic.

What powers it

Layer	Detail
Page	frontend/app/(app)/assets/page.tsx
API	GET/POST/PATCH/DELETE /api/assets, /assets/stats, /assets/:id/history, /assets/:id/maintenance
Backend	controllers/asset.controller.js on top of the generic CRUD factory
Model	models/Asset.js - one collection for all types, with a free-form attributes map
Notable	A pre-save hook derives the text status from the numeric condition, so the two can never disagree

Likely viva questions

- **Why one collection for seven asset types?** Because they share 90% of their fields. Seven near-identical collections would mean seven near-identical sets of queries and seven chances to get pagination wrong. Type-specific fields live in an attributes map instead.
- **What is the CRUD factory?** Eleven of the twelve modules need the same five operations - list with filter/sort/pagination, read, create, update, delete. Writing them per module would be around 700 lines of near-identical code. The factory in controllers/crud.factory.js generates them from a config, and each module adds only its genuinely specific handlers on top.
- **Why is deleting an asset a soft delete?** Its maintenance history and any work orders referencing it must stay intact. The asset is marked inactive rather than erased.

7. Biodiversity Management

/biodiversity - Module 4 - public; ecologist role to verify

What it is for

The species catalogue and the ecological mathematics computed over it. This is the most academically substantial module in the project, and the one worth understanding best before a viva - it is where the real ecology lives.

Tabs

Tab	What it shows and why it matters
Indices	Species richness, Shannon-Wiener, Pielou evenness, Simpson, Margalef, Berger-Parker dominance - each shown with its formula, its value for the current data, and a plain-English note on what it measures. Also shows the abundance vector the indices were computed from, and per-taxocene indices.
Catalogue	Species cards with conservation status, habitat, seasonality and images. Invasive and indicator species are badged. Opening one offers a GBIF cross-check against the global occurrence record.
Observations	Field sightings, filterable by verification state and source. Only VERIFIED observations count towards the indices - that gate stops one enthusiastic or mistaken reporter from moving a park's score.
Compare	Every park side by side on the same indices, so "which park needs conservation attention" has an evidence-based answer.
Index calculator	Type any abundance vector and the server recomputes every index live. Use this in the review: enter "100, 1, 1, 1" then "25, 25, 25, 25" - same richness, completely different evenness.

What powers it

Layer	Detail
Page	frontend/app/(app)/biodiversity/page.tsx
API	GET /api/biodiversity/indices, /compare, /seasonality, /species, /observations; POST /indices/preview
Service	services/biodiversity.service.js - all index mathematics
Models	models/Species.js (catalogue) and models/Observation.js (field records)
External	GBIF occurrence search and backbone taxonomy matching
Notable	Population counts are NOT stored on the species; they are aggregated from observations so the indices always reflect real field records

Likely viva questions

- **What does the Shannon index actually measure?** $H' = -\sum(p_i * \ln(p_i))$, where p_i is a species' share of all individuals. It is the uncertainty in guessing the species of a randomly drawn individual. It rises with both the number of species and how evenly they are spread. A single-species site scores 0.
- **Why report evenness separately?** Because richness alone lies. A park with 500 pigeons and one hawk has two species, the same as one with 250 of each - but they are not equally healthy. Pielou evenness $J' = H'/\ln(S)$ isolates that, so a park dominated by one invasive species cannot score well on species count alone.
- **What is the taxocene point?** Ecologists normally compute diversity within one taxonomic group surveyed by one method, not across all life. Pooling bird counts with plant-stem counts mixes units of survey effort. This project reports the pooled score because a manager needs one number per park, but it also reports per-class indices - and says so openly rather than hiding it.

- **Why do unverified sightings not count?** Because citizen data is unvalidated by definition. Verification by an ecologist is the gate where public input enters the scientific record.

8. AI Ecosystem Monitoring

/ai - Module 5

What it is for

Vision analysis for five tasks: tree disease, plant identification, wildlife recognition, waste detection and fire detection. Submit an image and get a prediction, a full class-probability vector, a severity rating and a recommended action - with automatic escalation to an incident when a finding is both dangerous and confident.

Tabs

Tab	What it shows and why it matters
Analyse	Pick a task, paste a URL or upload a photo, choose a park. Returns the winning label, the confidence, the complete softmax vector, and the escalation decision with its reasoning shown as three checks.
Detections	Every past inference as an image gallery. An ecologist can confirm or overturn each one - that is what builds the ground-truth set a retraining pipeline would consume.
Models	A model card per task: backbone, version, input size, and the full output class vocabulary with the severity each label implies.
Performance	Detection volume, mean confidence per model, confidence distribution, and - where human reviews exist - the observed precision.

What powers it

Layer	Detail
Page	frontend/app/(app)/ai/page.tsx
API	POST /api/ai/analyze, /ai/:id/review; GET /ai/tasks, /ai/gallery, /ai/stats
Service	services/ai-inference.service.js
Model	models/AiDetection.js - stores the full probability vector, not just the winner
Extension point	Set AI_MODEL_ENDPOINT and the same code path POSTs to a real served model instead

Likely viva questions

- **Is the AI real?** Answer honestly, because honesty is the strong answer here. The inference CONTRACT is fully real - class vocabulary, softmax, argmax, severity mapping, the escalation rule, storage of the full probability vector. The logits come from a deterministic hash of the image rather than trained weights. This was the scope agreed at the Week-6 assessor review, which explicitly asked that simulated responses be stated rather than overstated.
- **What would the real model be?** Transfer learning: decode, resize to 224x224, normalise to ImageNet statistics, MobileNetV2 or EfficientNet-B0 backbone frozen initially, global average pooling, dropout 0.2, dense head of C classes, softmax. Transfer learning because the few thousand labelled images a municipality can realistically gather are far too few to train from scratch but ample to fine-tune a head.
- **Why deterministic rather than random?** So a demonstration cannot be derailed by a re-roll. The same photo always gives the same answer, and the seeded records stay stable.
- **Explain the escalation rule.** A high or critical severity finding opens an incident automatically only above a configurable confidence floor (default 85%). Below it, the finding is queued for a human instead - because a false fire alarm is expensive.

9. Environmental Sensor Monitoring

/sensors - Module 6

What it is for

Live readings from 32 monitoring devices across six measurement types, each normalised onto a common 0-100 scale and screened by a three-detector statistical anomaly ensemble.

Tabs

Tab	What it shows and why it matters
Network health	Deployed, online, warning and offline counts. Offline sensors are excluded from scoring rather than counted as zero.
Normalised scores	Mean score per sensor type with a note explaining how each raw unit is mapped - this is the step that makes incompatible units comparable.
Device grid	One card per sensor: current reading, normalised score, status, battery, threshold breach and staleness. Filterable by type.
Detail drawer	Time series over 6h to 7 days with anomalous readings marked as points and warning thresholds drawn as reference lines - so "why is this in warning" is answerable by looking. Also lists every detected anomaly with its z-score and the ensemble's reasoning.

What powers it

Layer	Detail
Page	frontend/app/(app)/sensors/page.tsx
API	GET /api/sensors/live, /:id/readings, /:id/anomalies; POST /:id/readings (ingestion), /simulate
Services	sensor.service.js (generation + ingestion), anomaly.service.js (detection)
Models	models/Sensor.js (registry) and models/SensorReading.js (time series, approx. 3,000 rows)
External	Open-Meteo weather and air quality anchor the simulated baseline every 15 minutes

Likely viva questions

- **The sensors are not real - is that a problem?** The hardware layer was documented as conceptual from Week 4. What matters is that the INGESTION PATH is real: validation, persistence, anomaly detection, threshold alerting and cached current values all run exactly as they would with physical hardware. POST /api/sensors/:id/readings is the door a real gateway would use; the simulator calls the same service function internally. Replacing simulation with real hardware changes nothing below that endpoint.
- **How are readings generated?** $x_t = \alpha * x_{t-1} + (1-\alpha) * (\text{base} + A * \sin(2 * \pi * (h - \text{phase}) / 24)) + \text{noise}$. The AR(1) term makes the series drift rather than jump between independent samples; the sine term gives a daily cycle; noise is Gaussian via Box-Muller. Crucially, "base" is not a constant when online - it is anchored to live weather and CAMS air-quality data for that park's real coordinates.
- **How does anomaly detection work?** Three detectors vote. Z-score ($|z| > 3$) is cheap and interpretable but its standard deviation is inflated by the very outliers it seeks. The modified z-score uses median absolute deviation, which has a 50% breakdown point and so is not fooled by extreme values. Tukey's IQR fence assumes no distribution at all. A reading is flagged when at least two agree - majority voting cuts the false positives any single detector produces on noisy field data.
- **Why normalise readings?** Because a LOW AQI is good while a HIGH soil-moisture reading is good, and they are in different units entirely. Three shapes cover every sensor: higher-is-better, lower-is-better, and an optimal band (temperature and humidity). AQI is special-cased through the CPCB category map rather than linearly

rescaled, because stretching an already-piecewise index would distort its category boundaries.

10. Citizen Engagement Portal

/citizen - Module 7 - sign-in required to submit

What it is for

Where the public reports issues, logs wildlife sightings and gives feedback. The module implements a complete flow: citizen submits, officer reviews, and on acceptance the report becomes either a tracked incident or a verified biodiversity record.

Tabs

Tab	What it shows and why it matters
All reports	Every submission with status, upvotes and - where accepted - a link to the incident it became. That link is the point of the module: it closes the loop between reporting something and seeing it acted on.
My contributions	The signed-in citizen's own history, upvotes received, and how many submissions were acted on.
Community impact	Submissions by category and park, acceptance rate, and a top-contributors leaderboard.
Submit	Category, description, park, and location - with "use my location" via browser geolocation and reverse geocoding to a readable street address. Wildlife sightings can name a species.
Officer review	Accept, hold or reject. Accepting an issue requires a severity and an exposure estimate, because those feed the incident's triage score - the officer is not just saying yes, they are supplying the facts that determine queue position.

What powers it

Layer	Detail
Page	frontend/app/(app)/citizen/page.tsx
API	GET/POST /api/citizen/reports, /my-reports, /stats; POST /:id/upvote, /:id/review
Backend	controllers/citizen.controller.js - the review handler is where public input enters the record
Model	models/CitizenReport.js, linked to Incident and Observation
External	OpenStreetMap Nominatim for reverse geocoding

Likely viva questions

- **Why are reports separate from incidents?** Reports are unverified public input; incidents are operational work items officers act on. Merging them would put unvalidated data straight into the work queue. Keeping them separate, with a link on acceptance, preserves the audit trail from citizen to resolution.
- **Why do upvotes matter?** They are the community-signal term in the triage formula, weighted at 0.10. Upvoting a report re-scores its linked incident, so public attention genuinely moves things up the queue - but only slightly, so it can never outrank a hazard.
- **What is the acceptance rate metric for?** It measures engagement QUALITY, not volume. A thousand submissions that officers ignore is a worse outcome than fifty that get acted on.

11. Incident & Alert Management

/incidents - Module 8 - officer role

What it is for

The operational core. "Priority-based solving" is named as a differentiator in the project deck, so priority here is COMPUTED from the incident's own attributes rather than typed in by whoever filed the report.

Tabs

Tab	What it shows and why it matters
Triage queue	Open incidents in the order they should be worked, each showing its score and the five factor values behind it. Overdue incidents are flagged against their response target.
All incidents	Full history with filters, including resolved ones and their resolution times.
Alerts	The generated notification stream - sensor thresholds, anomalies, high-severity AI findings. Acknowledge individually or clear the board.
Statistics	Incidents by type (bar intensity follows hazard weight), by priority band, and mean response time against target per type.
Detail drawer	Full factor breakdown of the score, the status timeline, and the actions: assign an officer, raise a work order, resolve.

What powers it

Layer	Detail
Page	frontend/app/(app)/incidents/page.tsx
API	GET /api/incidents, /triage, /stats; POST /:id/assign, /:id/resolve, /:id/work-order
Service	services/priority.service.js (triage), alert.service.js (deduplicated alerts)
Models	models/Incident.js with an embedded timeline, models/Alert.js
Notable	The API never accepts a priority value from the client - it is always recomputed server-side

Likely viva questions

- **How is priority calculated?** $P = 100 * (0.35 * \text{hazard} + 0.25 * \text{severity} + 0.20 * \text{exposure} + 0.10 * \text{urgency} + 0.10 * \text{community})$. Hazard is the intrinsic danger of the incident type - a fire is 1.0, vandalism 0.4 - so no amount of upvotes can make a broken bench outrank a fire.
- **Why is exposure log-scaled?** Because the difference between 10 and 100 people affected matters far more than between 4,000 and 4,090. A linear term would let one very large number swamp everything else.
- **Explain the ageing term.** $U(t) = 1 - e^{-(t/\tau)}$, where τ is the incident type's target response time. An unattended incident must climb the queue or it starves behind newer, slightly-higher-scoring ones. But the curve saturates: at $t = \tau$ it has about 63% of the ageing weight, at $3 * \tau$ about 95%, and it never grows beyond that - so an old complaint can never outrank a new fire.
- **Why deduplicate alerts?** A sensor that stays out of range produces a reading every minute. Raising a new alert each time buries the operator. Alerts are keyed by the CONDITION, so a repeat bumps an occurrence counter instead of creating a new row - and the alert auto-resolves when the sensor returns to range.

12. Maintenance Management

/maintenance - Module 9 - officer role

What it is for

Work orders - the planned counterpart to incidents. Incidents are unplanned events; work orders are scheduled tasks. Completing one is what actually restores an asset's condition.

Tabs

Tab	What it shows and why it matters
Work orders	Filterable list with progress bars, priority and assignee. Overdue orders carry a red edge.
Calendar	A real month grid with orders on their scheduled days, colour-coded by status and navigable month to month.
Workload	Cost by work type, orders by status, and open assignments per person - an unbalanced column is the case for reallocating crew rather than hiring.
Detail drawer	Update progress with a slider. Setting 100% marks the order complete, appends it to the linked asset's maintenance history, and raises that asset's condition automatically.

What powers it

Layer	Detail
Page	frontend/app/(app)/maintenance/page.tsx
API	GET /api/maintenance, /calendar, /stats; PATCH /:id/progress
Backend	controllers/maintenance.controller.js
Model	models/WorkOrder.js - pre-save hook keeps progress, status and completion timestamp consistent
Notable	Completion writes to two collections at once, so the asset register and the maintenance log cannot drift apart

Likely viva questions

- **Why are work orders separate from incidents?** Different lifecycles. An incident is reported, triaged and resolved; a work order is scheduled, worked and completed. One incident may produce several work orders, and most work orders come from a maintenance schedule rather than an incident at all.
- **What happens when you complete one?** Three things in one operation: the order is marked complete with a timestamp, a maintenance record is appended to the linked asset, and the asset's condition rises. That is what stops the inventory staying permanently pessimistic after a repair.
- **How does an order become overdue?** A pre-save hook flags any scheduled order whose date has passed. It is derived, not a status somebody has to remember to set.

13. Analytics & Reports

/analytics - Module 10

What it is for

Cross-module trend analysis and data export. Read-only aggregation over the whole database, with a saved-report archive and CSV/PDF export.

Tabs

Tab	What it shows and why it matters
Overview	Headline figures for the selected window and park: health, biodiversity, incident resolution rate, maintenance completion, and the five sub-indices as bars.
Trends	Environmental indicators with anomaly counts overlaid; biodiversity with Shannon recomputed per month; incident volume against backlog; citizen participation stacked by category.
Compare	Every park on all five sub-indices side by side, plus a full comparison table.
Reports	Saved ecological assessments with findings and recommendations. Each stores a frozen metric snapshot, so a published report keeps showing the figures it was written against even after live scores move on. Exportable to PDF.
Export	Five datasets, each as CSV or PDF, respecting the current park filter.

What powers it

Layer	Detail
Page	frontend/app/(app)/analytics/page.tsx
API	GET /api/analytics/summary, /environmental-trend, /biodiversity-trend, /incident-trend, /engagement, /park-comparison, /export
Backend	controllers/analytics.controller.js - MongoDB aggregation pipelines with \$group and \$dateToString
Libraries	Recharts for charts; jsPDF + autoTable for client-side PDF generation

Likely viva questions

- **Where does the aggregation happen?** In MongoDB, using aggregation pipelines. Grouping thousands of readings by day in the database and returning 30 rows is far cheaper than sending every row to the browser to be grouped there.
- **Why is the biodiversity trend special?** Because Shannon is genuinely recomputed from each month's own observations, not averaged. Averaging an index across months is mathematically meaningless - the index is a property of a distribution, not a quantity you can take a mean of.
- **Why generate PDFs in the browser?** Server-side PDF rendering means shipping a headless Chrome into the deployment. jsPDF does it client-side from data the browser already has, and nothing leaves the machine.

14. AI Environmental Assistant

/assistant - Module 11

What it is for

A question-answering chatbot over the live database. Retrieval-augmented: the retrieval half is genuinely implemented with TF-IDF and cosine similarity, while answers are composed from templates rather than a language model.

Tabs

Tab	What it shows and why it matters
Conversation	Ask in plain language. Each answer shows the matched intent, the intent confidence and the response time.
Sources	The documents the retrieval step surfaced, with their similarity scores - so every claim is traceable to the record it came from.
Suggestions	Starter questions covering each supported intent.
How it works	The retrieval mathematics stated openly on the page itself.

What powers it

Layer	Detail
Page	frontend/app/(app)/assistant/page.tsx
API	POST /api/assistant/ask, /assistant/search; GET /assistant/suggestions
Service	services/assistant.service.js - tokenising, intent routing, TF-IDF index, answer composition
Model	models/ChatMessage.js - stores citations and intent per turn
Notable	The same retrieval powers the global search box in the navigation bar

Likely viva questions

- **Is this a real LLM?** No, and say so plainly. Retrieval is real: a corpus is rebuilt from MongoDB, weighted with tf-idf, and ranked by cosine similarity. Generation is template-based. The advantage is that every figure is read from the database and cited, so the assistant cannot hallucinate a number - which a language model at this scale absolutely would.
- **Explain TF-IDF.** $tf(t,d)$ is how often a term appears in a document, divided by document length. $idf(t) = \ln(N / (1 + n_t)) + 1$ downweights terms appearing everywhere - "park" carries almost no signal in a corpus of parks. The weight is $tf * idf$.
- **Why cosine and not dot product?** Document lengths here vary by an order of magnitude. A species description dwarfs an incident title, and without length normalisation the long documents would always win regardless of relevance. Cosine divides by both vector magnitudes.
- **How would you upgrade it?** Replace the TF-IDF stage with sentence embeddings and a vector database, then feed the retrieved documents to an LLM for generation. The interface - retrieve, then compose - would not change.

15. Administration

/admin - Module 12 - admin role only

What it is for

User management, system configuration, the audit trail, and platform maintenance actions. The consequential control here is the health-index weight editor.

Tabs

Tab	What it shows and why it matters
Users	Full CRUD over accounts with role assignment and home park. Public registration always creates a citizen; elevated roles can only be granted here.
Settings	The five health-index weights as sliders, validated to sum to 1.00 before saving, with a normalise button. Also the anomaly z-score threshold and the AI escalation confidence floor - the two algorithm parameters worth tuning in the field.
Audit log	Append-only record of every create, update and delete, with the changed fields shown as before-and-after. Filterable by action and entity.
System	Collection sizes, database and runtime status, live status of every external integration, and maintenance actions: recompute all scores, reindex the assistant, regenerate the demo dataset.

What powers it

Layer	Detail
Page	frontend/app/(app)/admin/page.tsx
API	GET/POST/PATCH/DELETE /api/admin/users; GET/PATCH /admin/settings; GET /admin/audit-log, /admin/stats
Backend	controllers/admin.controller.js - the whole router is behind requireRole("admin")
Models	models/User.js, models/Setting.js (single document), models/AuditLog.js

Likely viva questions

- **Why validate that weights sum to 1?** Because they define the composite index. Saving weights that sum to 1.3 would silently inflate every score in the system. The save is rejected, and on success every park is re-scored immediately - leaving some parks on the old formula and some on the new one would be worse than not allowing the change.
- **Why an audit log?** Municipal systems must answer "who changed this and when". It is append-only: corrections are new entries, never edits, so the history cannot be rewritten.
- **Can an admin lock themselves out?** No. The controller refuses to let an administrator change their own role or deactivate their own account.

16. The Mathematics, in Plain English

Every formula in the project, with what it means and why that form was chosen. This chapter and the tech stack chapter together cover the large majority of viva questions.

Ecosystem Health Index

```
EHl = sum(w_k * S_k) / sum(w_k)

w_air 0.25   w_water 0.20   w_soil 0.15
w_tree 0.20   w_biodiversity 0.20
```

A weighted mean of five sub-indices, each normalised to 0-100 where higher is better. Dividing by the sum of the weights actually present handles missing data correctly: a park with no soil sensor is scored on the four indicators it does have rather than being punished with a zero.

Normalising a sensor reading

```
higher-is-better: S = 100 * (x - min) / (max - min)
lower-is-better:  S = 100 * (max - x) / (max - min)
optimal band:     S = 100 * (1 - |x - centre| / halfwidth)
```

Three shapes cover every sensor. Temperature and humidity use the band form, because both too hot and too cold are bad. AQI is special-cased through the CPCB category map instead of a linear rescale, since it is already a piecewise-linear index and stretching it again would move its category boundaries.

Air Quality Index (CPCB method)

```
Sub-index for one pollutant, by linear interpolation:

I = ((I_high - I_low) / (C_high - C_low)) * (C - C_low) + I_low

Overall AQI = max(I_pm25, I_pm10, I_no2, I_so2, I_co, I_o3)
```

The MAXIMUM, not the mean - and this is a favourite examiner question. Air is only as clean as its worst pollutant; averaging would let one hazardous pollutant hide behind five clean ones. CPCB breakpoints are the Indian standard, which is why the demonstration city is Indian.

Biodiversity indices

For S species with abundances $n_1 \dots n_S$, total N, and share $p_i = n_i / N$:

```
Richness          S = number of distinct species
Shannon-Wiener    H' = -sum(p_i * ln(p_i))
Pielou evenness   J' = H' / ln(S)           range 0 to 1
Simpson           D = sum(p_i^2)           reported as 1 - D
Margalef          D_Mg = (S - 1) / ln(N)
Berger-Parker     d = max(p_i)

Composite score = 100 * (0.35*H_norm + 0.25*J' + 0.20*S_norm + 0.20*C)
```

- **Shannon** is information-theoretic - the uncertainty in guessing the species of a randomly drawn individual. It carries the largest weight because it is the only term reacting to richness and evenness at the same time.
- **Evenness** is separated out so a park dominated by one invasive species cannot score well on species count alone.

- **Margalef** corrects richness for sampling effort, so parks surveyed with different intensity stay comparable.
- **The conservation term C** rewards recording threatened species, because their presence is stronger evidence of habitat quality than a common species is.

Incident triage score

```
P = 100 * (0.35*H + 0.25*S + 0.20*E + 0.10*U + 0.10*C)

H hazard weight of the type    (fire 1.0 ... vandalism 0.4)
S severity, (s - 1) / 4 for s in 1..5
E exposure, ln(1 + people) / ln(1 + 5000), capped at 1
U urgency, 1 - e^(-t / tau)
C community, ln(1 + upvotes) / ln(1 + 200), capped at 1
```

Exposure and community signal are log-scaled so a single very large number cannot swamp the rest. Urgency saturates on a bounded curve, so an old complaint climbs quickly while genuinely late and then stops - it can never outrank a new fire.

Anomaly detection - three detectors, majority vote

```
Z-score          z = (x - mean) / stddev          flag |z| > 3
Modified z-score  M = 0.6745 * (x - median) / MAD    flag |M| > 3.5
Tukey IQR fence   outside [Q1 - 1.5*IQR, Q3 + 1.5*IQR]

Flagged when at least 2 of the 3 agree.
```

The constant 0.6745 is the 75th percentile of the standard normal, which makes median absolute deviation a consistent estimator of the standard deviation - so the 3.5 threshold is comparable to the z-score's 3. The modified z-score matters because the plain z-score's standard deviation is inflated by the very outliers it is looking for, while the median has a 50% breakdown point.

Softmax (AI module)

```
p_i = e^(z_i / T) / sum_j( e^(z_j / T) )

implemented as e^((z_i - max z)/T) for numerical stability
```

Converts raw scores into probabilities that sum to 1. Subtracting the maximum prevents overflow without changing the result. Temperature T below 1 sharpens the distribution, making the model more confident.

TF-IDF retrieval (assistant)

```
tf(t,d) = count of t in d / length of d
idf(t)   = ln( N / (1 + n_t) ) + 1
weight   = tf * idf

cos(q,d) = dot(q, d) / ( ||q|| * ||d|| )
```

Sensor generation

```
x_t = alpha * x_(t-1)
      + (1 - alpha) * (base + A * sin(2*pi*(h - phase) / 24))
      + noise

alpha = 0.7 (AR(1) persistence), noise ~ N(0, sigma) via Box-Muller
```

Real environmental variables have a daily cycle and are strongly autocorrelated - they drift rather than jumping between independent samples. When the machine is online, "base" is replaced by live weather and CAMS air-quality data for that park's real coordinates.

17. The Database

MongoDB with 16 collections. Documents rather than tables: a park is one document containing its geometry, cached scores and facilities, instead of being split across several joined tables.

Collection	What it holds
Park	The 6 parks. GeoJSON centre point and boundary polygon, cached scores, facilities.
Asset	231 physical assets of 7 types. Condition, maintenance history, type-specific attributes.
Species	30 species. Conservation status, habitat, seasonality, invasive and indicator flags.
Observation	420 field sightings. The abundance records every biodiversity index is computed from.
Sensor	32 devices. Registry, calibration bounds, thresholds, cached current value.
SensorReading	About 3,000 readings. The time series, with each reading's anomaly verdict.
CitizenReport	41 public submissions, linked to incidents or observations on acceptance.
Incident	34 incidents with a computed priority score and an embedded status timeline.
WorkOrder	38 scheduled maintenance tasks.
AiDetection	30 inference results, each with its full class-probability vector.
Alert	Generated notifications, deduplicated by condition.
EcoReport	Saved assessments with frozen metric snapshots.
User	12 accounts across the four roles.
AuditLog	Append-only change history.
Setting	Single document - index weights and algorithm parameters.
ChatMessage	Assistant conversation history with citations.

Design decisions worth defending

- **Observations are separate from Species.** Population counts are never stored on the species document; they are aggregated from observations. That way the indices always reflect real field records rather than a manually edited number that drifts.
- **Readings are separate from Sensors.** The reading collection is by far the largest and grows without bound, so it is kept deliberately narrow. Only the latest value is cached on the sensor.
- **Maintenance is embedded in Asset.** Records are always read together with their asset, are append-only, and are bounded in practice - so embedding avoids a join for no cost.
- **Timelines are embedded in Incident.** Same reasoning: the detail view needs the whole history, and it never grows large.
- **GeoJSON everywhere with 2dsphere indexes.** Makes proximity queries a single indexed operation and lets the map consume the data almost directly.

18. What This Project Does Not Do

Read this chapter carefully

Being able to state your own limitations precisely is the single strongest thing you can do in a review. An examiner who finds a weakness you did not mention will press it. An examiner you have already told will usually move on - and will trust everything else you said more.

Limitation	The honest position
No physical sensors	The hardware layer was documented as conceptual from Week 4. The ingestion path is real and a physical gateway would POST to the same endpoint; only the source of the numbers differs. Readings are anchored to live weather data where the network allows.
No trained vision models	The inference contract is fully implemented; the logits come from a deterministic surrogate. This was the scope agreed at the Week-6 assessor review, which asked that simulated responses be stated explicitly rather than overstated. Setting AI_MODEL_ENDPOINT swaps in a real served model with no other change.
The assistant is not an LLM	Retrieval is genuine TF-IDF with cosine ranking; generation is template-based. The trade-off is deliberate: templates cannot hallucinate a number, and every figure stays traceable to its record.
JWT stored in localStorage	Convenient for a prototype with a separate API origin, but vulnerable to XSS. A production deployment should use httpOnly cookies with SameSite=Lax and a CSRF token.
Pooled diversity index	Mixing bird counts with plant-stem counts mixes units of survey effort. The project reports the pooled score because a manager needs one number per park, and reports per-taxocene indices alongside it precisely because the pooled figure is imperfect.
No automated test suite	Verification was done with a 52-check end-to-end script against both live servers plus TypeScript compilation, rather than unit tests. Adequate for a prototype; a production system would want unit tests around the service layer especially.
Single-process cache	The external-API cache is an in-memory Map. Fine for one server; multiple instances would need Redis.

19. One-Page Cheat Sheet

If you remember only five things

- **The health score is a weighted mean of five sub-indices**, and dividing by the weights actually present is what makes missing sensors harmless.
- **AQI is the MAXIMUM of the pollutant sub-indices, not the mean**, because air is only as clean as its worst pollutant.
- **Shannon measures richness and evenness together**; evenness is reported separately so one dominant invasive species cannot hide behind a long species list.
- **Incident priority is computed, never entered**, and its ageing term saturates so an old complaint can never outrank a new fire.
- **The AI and the sensors are simulated, but their contracts are real**. Say this before you are asked.

Numbers worth knowing

Figure	Value
Modules	12, plus auth and integrations
Frontend routes	16 (plus an auto-generated 404 page)
API route handlers	134
MongoDB collections	16
Lines of code	about 29,000 across 179 files
Seeded data	6 parks, 30 species, 231 assets, 32 sensors, 3,000 readings, 420 observations, 34 incidents, 38 work orders, 41 reports, 30 AI detections
End-to-end checks passing	52 of 52
Public APIs needing no key	4 (Open-Meteo x2, GBIF, Nominatim)

Questions you should not be caught out by

- **"Why MongoDB and not SQL?"** The data is naturally nested, GeoJSON is first-class, and the schema evolved during the build. A production GIS platform with heavy spatial joins would justify PostGIS.
- **"Why Node on both sides?"** One language, shared validation schemas between client and server, and shared type definitions.
- **"What was hardest?"** Making the numbers defensible rather than decorative - choosing index weights that can be justified, handling missing sensors correctly, and getting the ageing curve to age without letting it dominate.
- **"What would you do next?"** Real hardware on the existing ingestion endpoint, a trained model on the existing AI_MODEL_ENDPOINT hook, embeddings plus an LLM for the assistant, and unit tests around the service layer.
- **"Show me it working."** Open the Index Calculator on the Biodiversity page and type two abundance vectors with the same richness but different evenness. It proves the mathematics is real in about fifteen seconds.

The most important advice

Do not claim more than the project does. Every simulated part of this system is clearly labelled in the interface itself, and that is an asset in a review, not a weakness. A student who says "the inference contract is real, the weights are a surrogate, and here is exactly where a trained model would plug in" sounds far more credible than one who claims a working CNN and then cannot answer how it was trained.